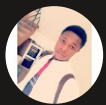


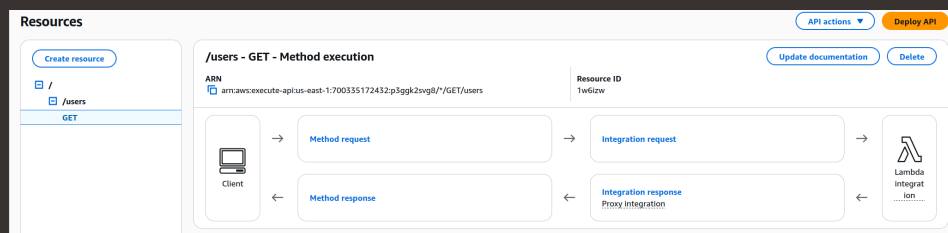


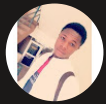
[nextwork.org](https://nextwork.org)

# APIs with Lambda + API Gateway



Ahmed Tetteh





# Introducing Today's Project!

In this project, I will demonstrate how to design the logic part of my application, i.e, the brains behind the processing of user actions such as clicking to application functionality.

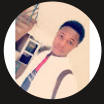
## Tools and concepts

Services I used were AWS Lambda and Amazon API Gateway. Key concepts I learnt include Lambda functions, creating a API to receive user requests and sending them to the Lambda function for processing, and finally, how to document an API.

## Project reflection

This project took me approximately 2 hrs. The most challenging part was learning to choose the appropriate API and configuration for my use case. It was most rewarding to see my API working effectively.

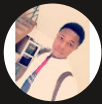
I chose to do this project today because it's part of the backend logic for my Three-tier web architecture.



## Lambda functions

AWS Lambda is a serverless resource that allows you to run code with provisioning or managing servers. I'm using Lambda in this project to create a backend system logic that runs whenever a request is made to the function. Hence, there is no need to have a server or VM running 24/7.

The code I added to my function first sets up the DynamoDBClient from the AWS SDK for Javascript. Then when the function is triggered, it extracts a userId from the queryStringParameters of the incoming event object. This ID is used as the unique key to look up a specific record in the DynamoDB table named. It returns a status of 200 if the UserId exists in the database, and 404 along with an error message if the UserId doesn't exist in the database.

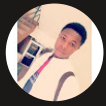


The screenshot displays the AWS Lambda console interface for a function named 'RetrieveUserData'. The 'Code source' tab is active, showing a JavaScript file named 'index.mjs'. The code is as follows:

```
1 // Import individual components from the DynamoDB client package
2 import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
3 import { DynamoDBDocumentClient, GetCommand } from '@aws-sdk/lib-dynamodb';
4
5 const ddbClient = new DynamoDBClient({ region: 'us-east-1' });
6 const ddb = DynamoDBDocumentClient.from(ddbClient);
7
8 async function handler(event) {
9   const userId = event.queryStringParameters.userId;
10   const params = {
11     TableName: 'UserData',
12     Key: { userId }
13   };
14
15   try {
16     const command = new GetCommand(params);
17     const { Item } = await ddb.send(command);
18     if (Item) {
19       return {
20         statusCode: 200,
21         body: JSON.stringify(Item),
22         headers: { 'Content-Type': 'application/json' }
23       };
24     } else {
25       return {
26         statusCode: 404,
27         body: JSON.stringify({ message: 'No user data found' }),
28         headers: { 'Content-Type': 'application/json' }
29       };
30     }
31   } catch (err) {
```

The left sidebar shows the 'EXPLORER' view with the file 'index.mjs' selected. Below it, there are 'DEPLOY' and 'TEST' buttons. The 'TEST' button is highlighted, and the 'TEST EVENTS' section is visible, showing 'NONE SELECTED'.



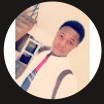


## API Gateway

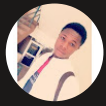
APIs are tools that act as middlemen between different resources, i.e., they enable them to talk each other. There are different types of APIs, like WebSocket APIs, REST API (standard web API), and HTTP API for routing requests. My API is a REST API, because it's a simple API that can be used by virtually any programming language, and provides everything we need for the backend logic.

Amazon API Gateway is simply the resource or tool that acts as the bridge between the user and the Lambda function for processing. It serves a middleman who carries the requests and responses from one end to another. I'm using API Gateway in this project because it comes with API management capabilities and authorization features that make an app more efficient - something Lambda does not have, and is not built for.

When a user makes a request, the API Gateway receives the requests, and then forwards them to the Lambda Function for processing. Lambda processes the request, then sends the response through the API Gateway back to the user.



The screenshot displays the AWS API Gateway console interface. At the top, a green notification bar states: "Successfully created REST API 'UserRequestAPI (p3ggk2vg8l)'". The left sidebar shows the navigation menu with "API Gateway" selected, and "Resources" is the active view under the "API: UserRequestAPI" section. The main content area is titled "Resources" and includes a "Create resource" button. Below this, the "Resource details" section shows the path "/" and the resource ID "aayaaq16rf". The "Methods" section, titled "Methods (0)", indicates that no methods are currently defined. Action buttons for "API actions" (including "API actions" and "Deploy API"), "Update documentation", and "Enable CORS" are visible in the top right. The "Methods" section also includes buttons for "Delete" and "Create method".

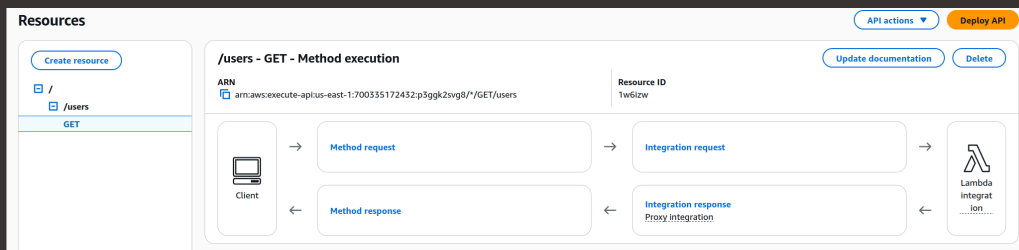
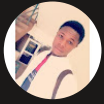


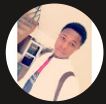
## API Resources and Methods

An API is made up of resources, which are simply individual endpoints within your API that handle different parts of its functionality.

Each resource consists of methods, which are the different actions that can be performed on a resource. These actions can include standard web interaction methods such as; 1. GET: to retrieve, 2. POST to add, 3. PUT to update, and 4. DELETE to remove data.

I created a GET method for my API resource, and connected it to my Lambda function. Now, when the function runs, Lambda will retrieve user data in a DynamoDB table.





# API Deployment

When you deploy an API, you deploy it to a specific stage. A stage is a snapshot of your API at a specific point in time. I deployed my API to the production stage.

To visit my API, I copied the "Invoke URL" link provided in my prod stage console. The API displayed an error because my DynamoDB table hasn't been set up yet, so, there is nothing to retrieve or return to the user.

Deploy API

Create or select a stage where your API will be deployed. You can use the deployment history to revert or change the active deployment for a stage. [Learn more](#)

Stage

\*New stage\*

Stage name

prod

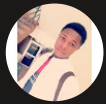
A new stage will be created with the default settings. Edit your stage settings on the **Stage** page.

Deployment description

This deployment serves live traffic to end-users

Cancel

Deploy

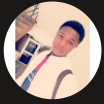


## API Documentation

For my project's extension, I am writing API documentation because after creating a functional API, it's also important to add a detailed description of the API's functionality, including its endpoints (e.g. /users), methods (e.g. GET), parameters (e.g. userId), and responses (e.g. errors or success response). It helps other developers understand how to use the API correctly and more efficiently. This can easily be done in the API Gateway console.

Once I prepared my documentation, I can publish it in a special file type like Swagger or OpenAPI for developers to easily use tools like SwaggerUI or Redoc to generate beautiful, interactive web pages about the API. You have to publish your API to a specific stage because the documentation has to be consistent with a specific version of the API deployed in that stage.

My published and downloaded documentation showed me many fields, including the documentation I created for the API, the API version, resources and methods.



```
EXPLORER
- AWS WITH LAMBDA + API GATEWAY
  - user_data.js
    - UserRequestAPI-prod-swagger-apigateway.json

10  "paths": {
11    "/users": {
12      "get": {
22        "x-amazon-apigateway-integration": {
29          },
30        "passthroughBehavior": "when_no_match",
31        "timeoutInMillis": 29000,
32        "responseTransferMode": "BUFFERED",
33        "contentHandling": "CONVERT_TO_TEXT",
34        "type": "aws_proxy"
35      }
36    }
37  },
38  },
39  "definitions": {
40    "Empty": {
41      "type": "object",
42      "title": "Empty Schema"
43    }
44  },
45  "x-amazon-apigateway-documentation": {
46    "version": "1",
47    "createdDate": "2026-03-07T14:21:29Z",
48    "documentationParts": [ {
49      "location": {
50        "type": "API"
51      },
52      "properties": {
53        "description": "The UserRequestAPI manages user data retrieval and manipulation. It supports operations to retrieve user details based on unique identifiers.",
54        "baseUrl": "https://p3gk2xvg8.execute-api.us-east-1.amazonaws.com/"
55      }
56    }
57  },
58  "x-amazon-apigateway-security-policy": "TLS_1_0"
59 }
```



[nextwork.org](https://nextwork.org)

# The place to learn & showcase your skills

Check out [nextwork.org](https://nextwork.org) for more projects

